Graph Neural Networks for Directed Acyclic Graphs Task Scheduling on Multi-CPU Architectures

Louis Lejaille
louis.lejaille@epitech.eu
2026

Abstract

Task scheduling on heterogeneous multi-CPU architectures represents a challenging combinatorial optimization problem, particularly when workflow dependencies are modeled as Directed Acyclic Graphs (DAGs). Previous supervised learning approaches based on dense Multi-Layer Perceptrons (MLPs) demonstrated poor structural generalization on unseen graph topologies due to the limitations of flattened vector representations.

In this work, we investigate Graph Neural Network (GNN) architectures for supervised scheduling policy learning through behavior cloning. Unlike previous dense scheduling models, the proposed approach directly represents workflow dependencies as graph-structured data using PyTorch Geometric. Task nodes encode execution-related features while graph edges represent dependency constraints inside the scheduling DAG.

The proposed architecture follows a hybrid representation strategy. The workflow DAG is processed using graph neural message-passing operations, while processor runtime states are encoded separately as global contextual feature vectors concatenated after graph pooling operations.

Two graph-based architectures are evaluated: Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs). In addition, a valid action masking strategy is introduced during inference in order to restrict action selection to executable task-CPU assignments and prevent invalid scheduling decisions within sparse combinatorial action spaces.

Experimental results demonstrate that graph-based scheduling representations significantly improve structural generalization compared to previous MLP-based approaches, particularly on unseen DAG scheduling environments. The proposed GCN models successfully reproduce heuristic scheduling policies across increasingly dense workflow graphs, while GAT-based models occasionally outperform heuristic scheduling decisions on unseen scheduling scenarios.

Although the proposed behavior cloning approach remains fundamentally bounded by the heuristic policy used during dataset generation, the obtained results demonstrate that preserving workflow topology and relational dependency information is essential for robust scheduling policy learning in graph-structured scheduling environments.

I. Introduction

Task scheduling on multiprocessor architectures represents a major optimization problem in modern distributed and parallel computing systems. Efficient scheduling strategies are essential for maximizing resource utilization, minimizing workflow completion time, and improving overall computational efficiency across heterogeneous computing infrastructures.

In many modern scheduling environments, workflow applications are naturally represented as Directed Acyclic Graphs (DAGs), where nodes correspond to computational tasks and directed edges encode dependency constraints between tasks. Such graph-based workflow representations are widely used in distributed systems, cloud computing, and heterogeneous scheduling research because execution order and dependency propagation strongly influence scheduling quality.

This work extends the previous study introduced in "Task Scheduling on Multi-CPU Graph Architectures using Supervised Policy Learning" [1], where a supervised learning scheduler based on dense Multi-Layer Perceptrons (MLPs) was investigated. Although the previous approach successfully reproduced heuristic scheduling behavior on known environments, the model demonstrated severe performance degradation on previously unseen DAG structures.

These observations suggest that flattened vector representations are insufficient for capturing the structural and relational properties of DAG scheduling environments. In particular, scheduling decisions strongly depend on task dependencies, execution topology, graph depth, and neighborhood interactions between tasks.

To address these limitations, this work investigates the use of Graph Neural Networks (GNNs) for topology-aware scheduling policy learning. Unlike dense neural architectures, GNNs are specifically designed to process graph-structured data through neighborhood aggregation and message-passing mechanisms, allowing the model to propagate information directly through workflow dependencies.

The proposed scheduling framework follows a hybrid representation strategy. Workflow dependencies are represented as graph-structured data and processed using graph neural networks, while processor runtime states are encoded sepa-

rately as global contextual feature vectors. This design allows the scheduler to jointly reason about workflow topology and processor availability during scheduling inference.

Several graph-based architectures are evaluated, including Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs), under progressively denser scheduling environments and increasingly complex DAG structures. The objective of this study is to evaluate whether graph-aware neural representations improve structural generalization capabilities compared to previous MLP-based scheduling approaches.

Experimental results demonstrate that graph-based scheduling models significantly improve generalization on unseen workflow topologies while maintaining stable scheduling performance under increasing DAG density and scheduling complexity.

II. Related Work

A. Heuristic-based DAG Scheduling

Task scheduling in heterogeneous multiprocessor and distributed systems has been extensively studied for many years. In modern scheduling environments, workflow applications are commonly represented as Directed Acyclic Graphs (DAGs), where nodes represent computational tasks and edges represent precedence constraints and data dependencies between tasks [2], [3]. Such DAG-based representations are widely used in cloud computing, Spark workflows, Grid systems, and distributed computing infrastructures because they naturally model complex execution pipelines and task relationships.

Classical heuristic approaches for DAG scheduling mainly rely on static scheduling strategies. Among the most widely used methods, list scheduling heuristics play a central role [3]. These approaches generally operate in two phases: a task prioritization phase, where tasks are ranked according to graph-based metrics, and a mapping phase, where tasks are assigned to available computational resources. The main objective of these heuristics is to minimize the overall workflow completion time, commonly referred to as the makespan.

Many heuristic scheduling approaches exploit priority-based mechanisms to determine the execution order of tasks. Several graph-related metrics have been proposed in the literature to estimate task importance inside the DAG. One of the most common metrics is the bottom-level (bLevel), which estimates the longest remaining execution path from a task to the exit node [3]. Other commonly used metrics include the Critical Path Length (CPL), the Task Parallelism Score (TPS), and the Data Locality Score (DLS) [2]. These metrics aim to prioritize tasks belonging to critical execution paths, maximize task parallelism, and reduce communication overhead between dependent tasks.

Scheduling DAG workflows becomes significantly more challenging in heterogeneous computing environments. In practical systems, computational nodes often exhibit different processing capabilities, memory capacities, and network performances [2], [3]. Consequently, efficient scheduling heuristics must consider not only task dependencies but also resource heterogeneity, communication cost, workload balancing, and data locality. Ignoring these factors may lead to poor resource utilization, network congestion, and increased workflow execution times.

Traditional heuristic schedulers often rely on static topological ordering strategies such as First-In First-Out (FIFO) execution or predefined rank-based ordering [3]. However, several studies have shown that static scheduling strategies may fail to efficiently exploit the DAG structure in highly dynamic distributed environments. In particular, runtime uncertainty, temporal unpredictability, fluctuating network conditions, and variable resource availability can strongly impact scheduling performance [3]. To address these limitations, more recent heuristic approaches introduced just-in-time scheduling mechanisms, where scheduling decisions are dynamically performed when tasks become ready for execution instead of being entirely determined before runtime.

Recent heuristic methods additionally attempt to maximize the number of ready tasks generated during workflow execution in order to improve resource utilization and execution parallelism [3]. Other approaches integrate node-aware scheduling strategies capable of considering node capacities, communication costs, and runtime workload distribution [2]. These adaptive heuristics aim to improve load balancing and reduce bottlenecks in heterogeneous infrastructures.

Despite their effectiveness and relatively low computational complexity, heuristic scheduling approaches still suffer from several limitations. Most heuristic methods rely on handcrafted rules and static decision-making processes, which reduce their ability to adapt to highly dynamic environments. Furthermore, heuristic schedulers often struggle to generalize across different workflow structures and heterogeneous resource configurations. These limitations have motivated the emergence of learning-based scheduling approaches, particularly Deep Reinforcement Learning (DRL) and Graph Neural Network (GNN)-based methods, which aim to learn adaptive scheduling policies directly from workflow and system states.

B. Limitations of MLP-based Approaches on DAG Scheduling

With the emergence of machine learning-based scheduling techniques, Multi-Layer Perceptrons (MLPs) have been increasingly adopted to improve scheduling decisions in heterogeneous multiprocessor systems [4]. Compared to traditional heuristic approaches, MLP-based models are capable of learning scheduling patterns directly from historical execution data and runtime system metrics. These approaches provide improved adaptability and enable data-driven scheduling decisions in dynamic environments.

However, despite their advantages, MLP-based approaches remain limited when applied to DAG-based task scheduling problems. In dependent-task scheduling, workflows are represented as Directed Acyclic Graphs (DAGs), where task dependencies and communication relationships play a critical role in determining scheduling efficiency [4]. Efficient scheduling decisions therefore require a precise understanding

of precedence constraints, critical paths, communication costs, and task interactions across heterogeneous resources.

Traditional MLP architectures primarily process tasks as independent feature vectors and do not explicitly model graph topology or structural dependencies between tasks. Consequently, MLP-based schedulers may fail to fully capture the relational information embedded in DAG workflows. In particular, they struggle to represent long-range task dependencies, multi-hop interactions, and dynamic communication patterns between distributed computational nodes.

Recent studies have shown that graph-structured learning methods are more suitable for dependent-task scheduling problems because they directly exploit the relational structure of DAG workflows [5]. Unlike MLPs, Graph Neural Networks (GNNs) perform message passing and neighborhood aggregation operations that allow nodes to exchange structural and contextual information. Such mechanisms enable GNN-based models to better capture task dependencies, execution order constraints, and communication-aware scheduling information.

Another important limitation of MLP-based approaches lies in their reliance on handcrafted feature engineering. Since MLPs do not naturally encode graph structures, scheduling systems often require manually designed features such as task priority scores, critical path metrics, communication costs, or resource utilization statistics [4]. This dependence on manually constructed representations may reduce model generalization across different workflow structures and heterogeneous computing environments.

Furthermore, modern multiprocessor scheduling environments are highly dynamic and heterogeneous. Runtime variations in workload distribution, processor utilization, communication latency, and resource availability continuously modify the execution conditions of workflows [4]. Under such conditions, fixed feature-based MLP models may struggle to adapt efficiently to unseen workflow topologies or rapidly changing execution states.

To overcome these limitations, recent research increasingly explores graph-based deep learning and reinforcement learning approaches for scheduling problems. In particular, GNN-based Deep Reinforcement Learning (DRL) methods have demonstrated promising results for computation task scheduling in dynamic distributed systems [5]. By combining topology-aware graph representations with adaptive policy learning, these approaches aim to learn scheduling strategies capable of dynamically optimizing task allocation while considering both workflow dependencies and heterogeneous resource constraints.

C. Graph Neural Networks for Graph-Structured Scheduling

Recent advances in task scheduling research increasingly rely on Graph Neural Networks (GNNs) to better exploit the structural properties of Directed Acyclic Graphs (DAGs) [5], [6]. Unlike traditional Multi-Layer Perceptrons (MLPs), which process scheduling states as flat feature vectors, GNNs are specifically designed to operate on graph-structured data by propagating information through node connections using message-passing mechanisms [7]. This property makes GNNs particularly suitable for scheduling problems where both task dependencies and hardware communication topologies naturally form graphs.

In DAG-based scheduling environments, workflow applications are represented as graphs where nodes correspond to computational tasks and edges encode precedence constraints and communication dependencies [1], [6]. Similarly, modern heterogeneous architectures can also exhibit relational structures through processor communication topology and resource interactions [1]. Such graph-based representations allow scheduling systems to explicitly model relationships between tasks, processors, communication paths, and execution constraints.

Several recent studies demonstrated that flat vector representations used by MLP-based schedulers fail to generalize efficiently on unseen graph topologies [1]. Experimental results showed that MLP architectures can successfully reproduce heuristic scheduling behavior on known scheduling scenarios, but their performance strongly degrades when exposed to new workflow structures or heterogeneous processor configurations [1]. These limitations are mainly caused by the inability of dense neural networks to explicitly capture relational dependencies, critical paths, and structural variations inside dynamic DAG environments.

Graph Neural Networks address these limitations by leveraging neighborhood aggregation and graph message-passing operations [7]. Instead of processing tasks independently, GNNs iteratively aggregate contextual information from neighboring nodes, enabling the model to learn topology-aware task representations. Such representations naturally encode dependency structures, communication relationships, task criticality, and execution context. Consequently, GNN-based schedulers generally exhibit stronger relational generalization capabilities compared to classical dense architectures [7].

Recent work combining GNNs with Deep Reinforcement Learning (DRL) further demonstrated the effectiveness of graph-aware scheduling policies in dynamic distributed environments [5]. In autonomous multi-robot systems, GNN-enhanced DRL schedulers model task-resource allocation problems as graph matching problems between computational jobs and heterogeneous robotic resources. By aggregating node features through attention-based message passing, these methods can dynamically capture structured relationships between tasks and available resources while adapting scheduling decisions to runtime system states [5].

Graph-based scheduling methods have also been explored for energy-aware cloud scheduling [6]. Recent studies showed that GNN-based DRL schedulers can learn topology-aware scheduling policies capable of jointly optimizing workflow completion time and energy consumption. In these approaches, DAG topology becomes a central scheduling signal, where structural properties such as graph depth, parallelism width, fan-out, critical path length, and communication patterns directly influence scheduling decisions [6]. Experimental results demonstrated that exploiting workflow topology through GNN

embeddings significantly improves scheduling quality compared to topology-agnostic approaches.

However, despite their strong performance, recent studies also highlighted several limitations of GNN-based scheduling systems [6]. In particular, GNN schedulers remain sensitive to out-of-distribution (OOD) conditions and structural distribution shifts. Significant performance degradation may occur when deployment workflows exhibit graph topologies that differ from those observed during training [6]. Changes in DAG depth, branching patterns, graph width, or resource heterogeneity may disrupt message-passing mechanisms and reduce policy generalization capabilities.

Furthermore, graph learning research highlighted that graph topology itself strongly impacts neural message propagation and representation quality [7]. Heterophilic graph structures, noisy graph connectivity, and dynamically changing node relationships may negatively affect neighborhood aggregation and reduce the robustness of graph-based models. These observations suggest that while GNNs provide major improvements over flat feature-based architectures, designing robust graph-aware scheduling policies capable of generalizing across heterogeneous workflow structures remains an open research challenge.

Overall, the integration of GNNs into scheduling systems represents a major evolution in DAG-based task scheduling research. By directly exploiting graph topology and relational dependencies, GNN-based approaches provide more expressive scheduling representations than traditional heuristics or MLP-based models. Combined with DRL, these methods offer adaptive and topology-aware scheduling strategies capable of dynamically optimizing task allocation in heterogeneous distributed environments.

Overall, existing scheduling approaches exhibit important limitations when dealing with dynamic DAG-based environments and heterogeneous graph structures. Classical heuristic methods rely heavily on handcrafted rules and static priority metrics, while MLP-based approaches fail to explicitly capture relational dependencies and graph topology. Although recent GNN-based scheduling methods demonstrated promising results, current approaches still face important challenges regarding structural generalization, robustness to topology shifts, and adaptability to unseen scheduling configurations.

Motivated by these observations, this work investigates a graph-aware scheduling approach designed for heterogeneous multi-CPU graph architectures.

III. Graph Representation and Neural Scheduling Architectures

A. Task Graph Encoding

The proposed scheduling framework models workflow applications as Directed Acyclic Graphs (DAGs), where nodes correspond to computational tasks and directed edges represent precedence constraints between tasks [1], [2], [6]. DAG-based representations are widely adopted in heterogeneous scheduling research because they naturally encode execution dependencies, communication relationships, and workflow topology.

Each task node is represented through a feature vector describing both structural and runtime scheduling information. The node feature representation includes:

- task execution duration,
- task execution state,
- number of dependencies,
- number of dependent tasks,
- task readiness status.

These features provide the graph neural network with both local task information and execution context required for scheduling decisions. Unlike flat vector representations traditionally used by MLP-based schedulers, graph-based representations explicitly preserve relational dependencies between tasks [4]. This allows the scheduling model to directly exploit structural information such as dependency chains, critical paths, graph depth, and task neighborhood interactions.

B. Edge Representation

Task dependencies inside the workflow are represented using directed graph edges stored through the `edge_index` representation of PyTorch Geometric. Each directed edge encodes a precedence relationship between two tasks, noted as `u -> v`, meaning that task `v` cannot begin execution before task `u` has completed.

This sparse graph representation efficiently models workflow topology while preserving dependency constraints required during scheduling execution [3]. The graph connectivity therefore becomes a direct scheduling signal used by the neural network during message propagation and neighborhood aggregation.

Preserving DAG topology is particularly important in heterogeneous scheduling environments since graph structure strongly influences execution parallelism, communication cost, and critical path propagation [6].

C. CPU State Encoding

In addition to the workflow DAG, processor runtime states are incorporated into the scheduling model [1]. Instead of representing processors as a graph structure processed through graph convolutions, CPU information is encoded as a global contextual feature vector.

Each processor contributes runtime scheduling information including:

- CPU availability,
- remaining execution time,
- current workload state.

These processor features are concatenated after graph pooling operations in order to combine workflow topology representations with global processor execution context.

This hybrid representation allows the model to jointly reason about:

- workflow dependency structures,

- task execution context,
- processor availability,
- runtime scheduling constraints.

D. Graph Construction Pipeline

Unlike previous vector-based scheduling approaches relying on manually flattened feature representations, the proposed framework directly constructs graph-based scheduling states compatible with PyTorch Geometric [1].

Each scheduling state is represented as:

$$G = (X, E, Y) \tag{1}$$

where:

- X represents node features,
- E represents graph connectivity,
- Y represents the target heuristic scheduling action.

The graph construction pipeline follows four main stages:

- Task DAG generation,
- node feature extraction,
- edge construction,
- PyTorch Geometric graph creation.

The resulting graph objects are then used as inputs for graph neural network training and inference.

E. Graph Convolutional Network (GCN)

The first proposed architecture relies on Graph Convolutional Networks (GCNs), a class of graph neural networks designed to perform convolution operations directly on graph-structured data [7]. Unlike classical dense neural networks, GCNs iteratively aggregate neighborhood information through message-passing operations, allowing nodes to integrate contextual information from connected neighbors.

This property is particularly important for DAG-based scheduling problems because scheduling decisions strongly depend on task dependencies, critical execution paths, and workflow topology [5], [6].

The proposed GCN architecture is composed of two graph convolution layers:

- `GCNConv(5 -> 16)`,
- ReLU activation,
- `GCNConv(16 -> 16)`,
- ReLU activation.

After graph convolutions, node embeddings are aggregated using a `global_mean_pool` operation in order to extract a global workflow representation summarizing the macroscopic state of the DAG.

Processor runtime features are then concatenated with this pooled graph representation before a final linear layer predicts action logits over the complete scheduling action space.

Consequently, the architecture does not independently evaluate each task embedding during inference. Instead, the pooled graph representation and global CPU context are jointly processed through a static linear prediction layer producing scheduling probabilities for all possible `(task, cpu)` assignments.

The architecture pipeline is summarized as:

- Task Graph,
- GCNConv,
- ReLU,
- GCNConv,
- ReLU,
- Global Mean Pool,
- CPU Feature Concatenation,
- Linear Layer,
- Action Logits.

The model is trained using supervised behavior cloning from heuristic scheduling trajectories with a Cross-Entropy loss function and the Adam optimizer.

A graph convolution layer updates node representations by aggregating neighborhood information:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \frac{1}{c_{uv}} W^{(l)} h_u^{(l)}\right) \tag{2}$$

where $h_v^{(l)}$ represents the node embedding at layer l, $W^{(l)}$ denotes the learnable weight matrix, and $\mathcal{N}(v)$ corresponds to the neighborhood of node v.

Although this architecture preserves workflow topology during message propagation, the final pooling operation compresses the DAG into a single global embedding before action prediction. As a result, action selection remains partially dependent on fixed output-space representations rather than direct node-level task evaluation.

F. Valid Action Masking

One of the main challenges of graph-based scheduling environments lies in the size of the action space. At each simulation step, the scheduler predicts an assignment action represented as `(task, cpu)`.

In dense DAG scheduling environments, the number of possible actions rapidly becomes very large, while only a small subset satisfies the scheduling constraints imposed by the environment.

Many actions are invalid because:

- the task has already been completed,
- task dependencies are not yet satisfied,
- the selected CPU is currently occupied.

Without additional constraints, the model frequently collapses toward invalid or non-productive decisions such as repeated `WAIT` actions.

To address this issue, a valid action masking strategy was introduced during inference. Let A denote the complete action space and $A_{valid}(s) \subseteq A$ the subset of executable actions under the current environment state s.

The final scheduling action is selected as:

$$a^* = \arg \max_{a \in A_{valid}(s)} \pi(a|s) \tag{3}$$

where $\pi(a|s)$ denotes the policy output logits produced by the neural network.

Importantly, the masking mechanism does not provide additional scheduling knowledge to the model. The scheduler must still learn:

- which task should be selected,
- which CPU should execute the task,
- under which scheduling context the decision is optimal.

The masking process simply enforces environment constraints during decision selection, similarly to legal move constraints used in board game environments.

This mechanism strongly stabilizes scheduling inference in large combinatorial action spaces while preserving the learning objective of the graph-based scheduling policy.

IV. Experimental Setup

A. Training Configuration

All experiments were implemented using PyTorch and PyTorch Geometric on GPU-accelerated environments. The proposed graph-based scheduling models were trained using supervised behavior cloning from heuristic scheduling policies [1].

The initial Graph Convolutional Network (GCN) architecture was composed of two `GCNConv` layers followed by ReLU activation functions and a global mean pooling operation. CPU state features were concatenated with the graph embedding before the final linear classification layer.

The training configuration was defined as follows:

- optimizer: Adam,
- learning rate: 0.001,
- loss function: CrossEntropyLoss,
- batch size: 64,
- number of epochs: 100,
- dataset size: 1000 seeds.

The initial D1 training process completed in approximately 2.28 minutes on GPU hardware [1].

B. Training Convergence Across Environment Complexities

The evolution of the training loss was analyzed across the three environment configurations in order to evaluate the stability of the proposed GCN behavior cloning scheduler under increasing scheduling complexity.

As DAG density, processor graph size, and action-space combinatorics increase, the optimization problem becomes progressively more difficult. Nevertheless, all training configurations exhibit stable convergence behavior without significant oscillation or divergence during optimization.

1) D1 Training Environment: Figure 1 illustrates the training loss evolution of the proposed GCN scheduler on the D1 environment. The model rapidly converges during the first training epochs before progressively stabilizing toward a final loss value close to 0.053.

The smooth convergence behavior indicates that the graph neural network successfully learned the heuristic scheduling policy on relatively small DAG scheduling environments.

2) D2 Training Environment: To evaluate scalability, the D2 environment introduced denser DAG structures, larger CPU graphs, and significantly larger action spaces.

Figure 2 shows that the optimization process remains stable despite the increased environment complexity. The final loss converges around 0.072, which is higher than the D1 configuration due to the increased scheduling difficulty and graph density.

However, the model still demonstrates stable learning dynamics and progressive convergence throughout training.

3) D3 Training Environment: The D3 configuration further increased scheduling complexity by generating substantially denser DAGs, larger task counts, increased dependency connectivity, and noisier heuristic scheduling data.

As shown in Figure 3, the optimization process remains stable even under highly dense scheduling conditions. The model converges toward a final loss value close to 0.074, indicating that the graph-based scheduling architecture scales consistently across increasingly difficult scheduling environments.

Although the optimization difficulty increases with environment density, no significant instability or divergence is observed during training.

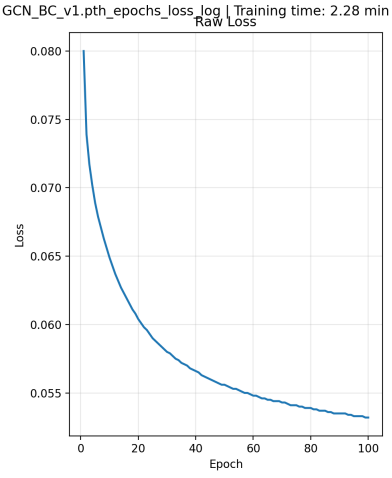

Fig. 1: Raw training loss evolution of the proposed GCN scheduler on the D1 environment.

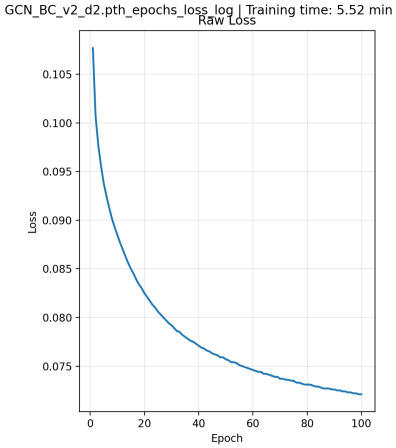

Fig. 2: Raw training loss evolution of the proposed GCN scheduler on the denser D2 environment.

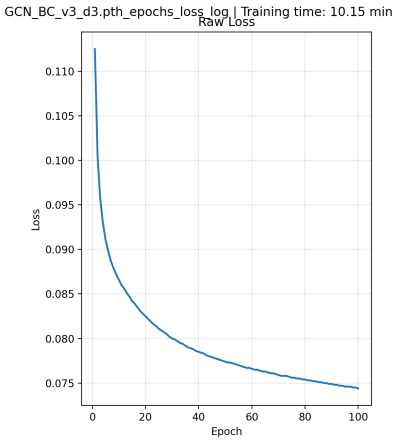

Fig. 3: Raw training loss evolution of the proposed GCN scheduler on the highly dense D3 environment.

C. Dataset Generation

The dataset was generated using deterministic simulation seeds in order to produce reproducible scheduling environments. Each seed generated:

- a unique task DAG,
- processor graph topologies,
- task durations,
- dependency structures,
- scheduling trajectories.

For each simulation state, the heuristic scheduling policy generated a target action represented as a `(task, cpu)` assignment. These heuristic decisions were then used as labels for behavior cloning training.

The graph dataset construction followed the graph representation introduced in the previous section:

$$G = (X, E, Y) \tag{4}$$

where node features X, graph connectivity E, and heuristic target actions Y were stored using PyTorch Geometric graph objects.

D. Heuristic Teacher Policy

The behavior cloning dataset was generated using a handcrafted heuristic scheduling policy designed to prioritize both task criticality and CPU locality.

At each scheduling step, the heuristic first selects executable tasks whose dependency constraints are fully satisfied. Among the ready tasks, a priority score is computed as:

$$S_{task}(t) = |\text{dependents}(t)| + 0.5 \times \text{duration}(t) + \epsilon \tag{5}$$

where:

- $|\text{dependents}(t)|$ represents the number of dependent tasks,
- $\text{duration}(t)$ corresponds to task execution time,
- ϵ is a small stochastic noise term.

This scoring strategy implicitly prioritizes tasks that:

- unlock larger portions of the DAG,
- belong to potentially critical execution paths,
- exhibit longer execution durations.

Once the task is selected, the heuristic assigns the task to an available CPU according to local neighborhood workload conditions. CPU selection minimizes the number of busy neighboring processors in the CPU graph topology:

$$S_{cpu}(c) = -|\text{busy\_neighbors}(c)| + \epsilon \tag{6}$$

This locality-aware scheduling mechanism aims to reduce local congestion while distributing workload across the processor graph.

Importantly, the heuristic remains relatively simple and does not explicitly optimize global DAG properties such as communication cost, long-range dependency propagation, or future scheduling states. Consequently, the generated behavior cloning dataset may contain locally optimal but globally suboptimal scheduling trajectories.

E. Environment Complexity Levels

To evaluate the scalability and robustness of graph-based scheduling models, several environment configurations with increasing complexity were introduced throughout the experiments.

1) D1 Environment: The initial environment configuration used relatively small DAGs and processor graphs:

- `MIN_GRID_SIZE = 2`
- `MAX_GRID_SIZE = 3`
- `MIN_TASKS = 10`
- `MAX_TASKS = 25`

This first environment was mainly designed to validate whether the GCN model could successfully reproduce heuristic scheduling behaviors through behavior cloning.

2) D2 Environment: To increase scheduling complexity, denser DAG structures and larger CPU topologies were introduced:

- `MIN_GRID_SIZE = 3`
- `MAX_GRID_SIZE = 5`
- `MIN_TASKS = 25`
- `MAX_TASKS = 60`

This second environment consistently increased:

- DAG density,
- graph connectivity,
- scheduling combinatorics,
- action space size.

The training duration for this configuration increased to approximately 5.52 minutes.

3) D3 Environment: To further investigate scalability and robustness, a third environment configuration introduced:

- larger DAGs,
- denser dependency structures,
- larger CPU graphs,
- increased heuristic stochasticity.

The task count was increased to:

- `MIN_TASKS = 50`
- `MAX_TASKS = 100`

The maximum number of DAG parents per node was also significantly increased in order to generate denser dependency graphs. Additionally, stochastic noise during heuristic dataset generation was increased from a multiplier of `0.01` to `0.05`.

This configuration produced significantly more complex scheduling scenarios and increased training time to approximately 10.15 minutes.

F. Evaluation Protocol

The proposed models were evaluated on both known and unseen environments.

Known environments corresponded to scheduling scenarios generated from seeds used during training. Unseen environments were generated from entirely different seed ranges in order to evaluate graph-aware generalization capabilities.

For example:

- training seeds: $0 \rightarrow 999$,
- unseen evaluation seeds: $1042 \rightarrow 1092$.

Performance was primarily evaluated using workflow completion time (makespan), measured as the total simulation time required to complete all tasks inside the DAG.

For each experiment, the following metrics were collected:

- average total execution time,
- minimum execution time,
- maximum execution time,
- number of scenarios outperforming heuristic scheduling.

V. Experimental Results

A. MLP vs GCN Generalization

One of the primary objectives of this work was to evaluate whether graph-based neural architectures provide better structural generalization capabilities than classical dense neural networks for DAG scheduling problems.

The first experiments compared the proposed Graph Convolutional Network (GCN) scheduler against the previous MLP-based scheduling approach on previously unseen environments.

The initial GCN model was trained on 1000 deterministic scheduling seeds using supervised behavior cloning from heuristic scheduling trajectories. Evaluation was then performed on unseen environments generated from entirely different seed ranges.

On known environments, the heuristic scheduler obtained:

- Average makespan: 66.64
- Minimum: 28
- Maximum: 117

while the GCN scheduler achieved:

- Average makespan: 67.16
- Minimum: 28
- Maximum: 117

These results indicate that the proposed GCN architecture successfully reproduces heuristic scheduling behaviors with very limited performance degradation.

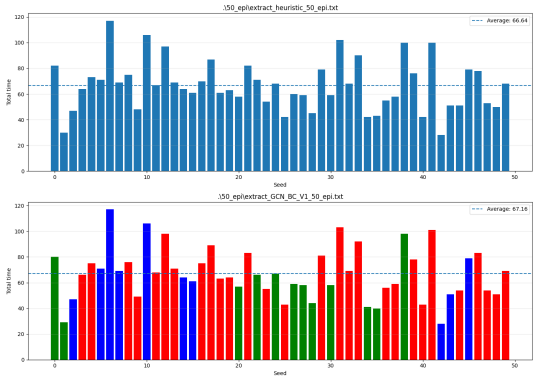

Fig. 4: Comparison between heuristic scheduling and the proposed GCN scheduler on known D1 environments.

The most important observation emerged during evaluation on unseen environments.

The heuristic scheduler obtained:

- Average makespan: 66.36
- Minimum: 28
- Maximum: 114

while the GCN model achieved:

- Average makespan: 66.98
- Minimum: 27
- Maximum: 115

The results remain remarkably close despite the fact that the evaluated DAG structures were never observed during training.

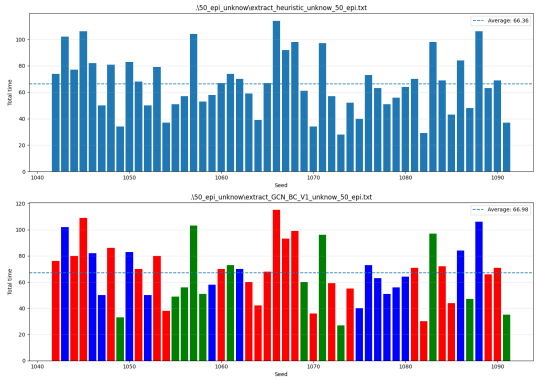

Fig. 5: Comparison between heuristic scheduling and the proposed GCN scheduler on unseen D1 environments.

In contrast, the previous MLP-based scheduling model exhibited severe generalization failure on unseen environments.

The unseen evaluation produced:

- GCN average: 68.04
- MLP average: 146.98

This substantial performance gap strongly suggests that preserving graph topology is essential for robust scheduling generalization.

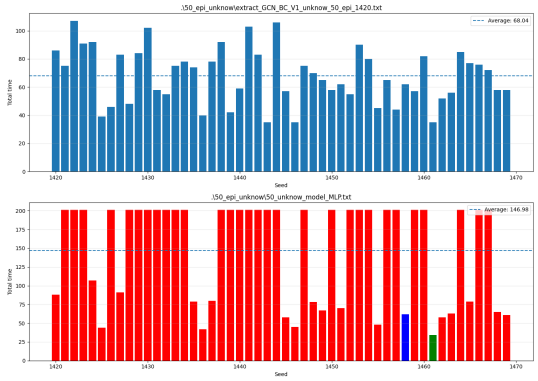

Fig. 6: Comparison between the previous MLP scheduler and the proposed GCN scheduler on unseen environments.

The experiments validate the central hypothesis of this work: graph neural networks provide substantially better structural inductive bias than dense neural networks for DAG scheduling problems.

B. GCN Scaling Experiments

To evaluate scalability, progressively denser scheduling environments were introduced throughout the experiments.

The D2 environment significantly increased:

- DAG density,
- CPU graph complexity,
- scheduling combinatorics,
- action-space size.

Despite the increased scheduling complexity, the GCN scheduler continued to reproduce heuristic scheduling behaviors with strong accuracy.

The heuristic scheduler obtained:

- Average makespan: 162.10
- Minimum: 87
- Maximum: 279

while the GCN scheduler achieved:

- Average makespan: 162.64
- Minimum: 86
- Maximum: 279

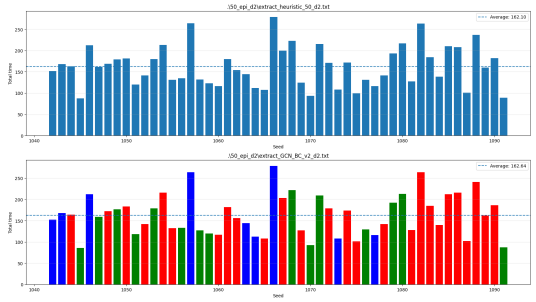

Fig. 7: Comparison between heuristic scheduling and the proposed GCN scheduler on denser D2 environments.

These results demonstrate that graph-based scheduling representations remain stable even as workflow density and scheduling complexity significantly increase.

To further investigate scalability, the D3 environment introduced:

- substantially denser DAGs,
- larger CPU graphs,
- increased dependency connectivity,
- noisier heuristic scheduling trajectories.

The heuristic scheduler obtained:

- Average makespan: 175.34
- Minimum: 123
- Maximum: 272

while the GCN scheduler achieved:

- Average makespan: 179.42
- Minimum: 135
- Maximum: 272

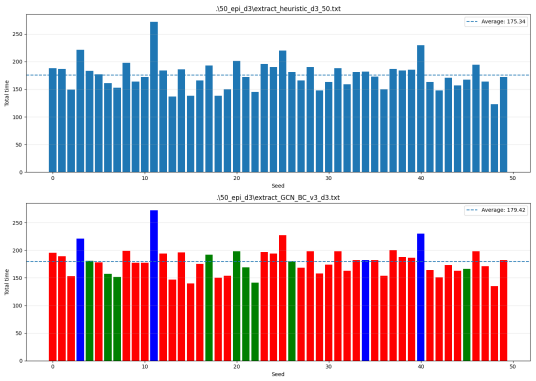

Fig. 8: Comparison between heuristic scheduling and the proposed GCN scheduler on highly dense D3 environments.

Although the GCN model does not outperform the heuristic scheduler, the experiments demonstrate that the proposed architecture successfully learns meaningful scheduling patterns even under highly dense DAG configurations.

Importantly, these observations also reveal a fundamental limitation of behavior cloning approaches. Since the model directly imitates heuristic scheduling decisions, the learned policy remains fundamentally bounded by the quality of the heuristic teacher policy used during dataset generation.

C. GAT vs GCN Scheduling Architectures

After evaluating Graph Convolutional Networks (GCNs), additional experiments investigated Graph Attention Networks (GATs) in order to evaluate whether attention-based neighborhood aggregation improves scheduling performance on dense DAG structures.

The GAT architecture replaced `GCNConv` layers with `GATConv` layers while preserving the overall graph scheduling pipeline.

On known D3 environments, the heuristic scheduler obtained:

- Average makespan: 175.34
- Minimum: 123
- Maximum: 272

while the GAT scheduler achieved:

- Average makespan: 179.44
- Minimum: 128
- Maximum: 272

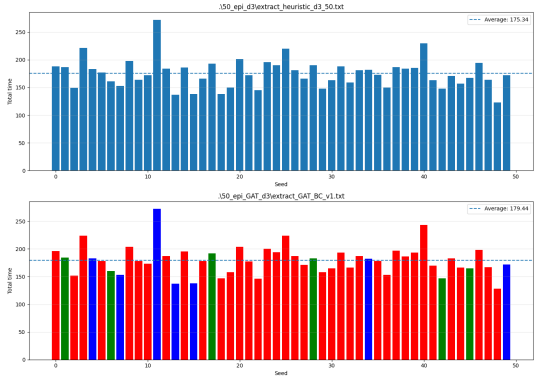

Fig. 9: Comparison between heuristic scheduling and the proposed GAT scheduler on known D3 environments.

The most interesting observations appeared during evaluation on unseen environments.

The heuristic scheduler obtained:

- Average makespan: 181.66
- Minimum: 138
- Maximum: 247

while the GAT scheduler achieved:

- Average makespan: 179.44
- Minimum: 128
- Maximum: 272

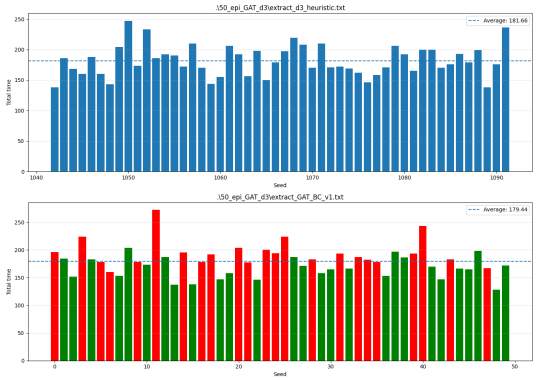

Fig. 10: Comparison between heuristic scheduling and the proposed GAT scheduler on unseen D3 environments.

Interestingly, the GAT model outperformed the heuristic scheduler on: 27 / 50 unseen scheduling scenarios.

These observations suggest that attention-based graph aggregation mechanisms may capture important dependency structures and critical scheduling relationships not explicitly represented inside heuristic scheduling policies.

Compared to GCNs, GATs dynamically weight neighborhood importance during message propagation, potentially allowing the model to prioritize:

- critical path tasks,
- communication bottlenecks,
- highly constrained dependency structures.

D. Known vs Unknown Environment Evaluation

Additional large-scale experiments were performed on 500 unseen scheduling scenarios in order to evaluate the robustness of the proposed GAT scheduler.

The heuristic scheduler achieved:

- Average makespan: 182.30
- Minimum: 117
- Maximum: 272

while the GAT scheduler obtained:

- Average makespan: 185.06
- Minimum: 124
- Maximum: 276

The GAT scheduler outperformed the heuristic scheduler on: 116 / 500 evaluation scenarios.

Although the overall average performance remained slightly below the heuristic baseline, these results demonstrate that graph attention mechanisms occasionally discover scheduling strategies superior to the heuristic teacher policy on specific workflow topologies.

E. Statistical Analysis

Across all experiments, several consistent observations emerge.

First, graph-based neural schedulers demonstrate substantially stronger generalization capabilities than classical dense neural networks on unseen DAG scheduling environments.

Second, both GCN and GAT architectures remain stable as workflow density and scheduling complexity increase significantly.

Finally, the experiments reveal an important limitation of supervised behavior cloning approaches. Since the models learn directly from heuristic scheduling trajectories, their performance remains fundamentally constrained by the quality of the heuristic teacher policy.

Table I summarizes the main experimental results obtained across all evaluated scheduling configurations.

TABLE I: Global comparison of scheduling performances across all experiments.

Model	Env	Eval	Avg	Min	Max	Better
Heuristic	D1	K	66.64	28	117	-
GCN V1	D1	K	67.16	28	117	Near
Heuristic	D1	U	66.36	28	114	-
GCN V1	D1	U	66.98	27	115	Strong Gen.
MLP BC	D1	U	146.98	-	-	Failure
Heuristic	D2	K	162.10	87	279	-
GCN V2	D2	K	162.64	86	279	Near
Heuristic	D3	K	175.34	123	272	-
GCN V3	D3	K	179.42	135	272	Stable
GAT V1	D3	K	179.44	128	272	Comparable
Heuristic	D3	U	181.66	138	247	-
GAT V1	D3	U	179.44	128	272	27/50
Heuristic	D3-L	U	182.30	117	272	-
GAT V1	D3-L	U	185.06	124	276	116/500

Several important conclusions can be drawn from these statistical observations.

First, the previous MLP-based scheduling model exhibits severe performance degradation on unseen DAG topologies, reaching an average makespan of 146.98 on unknown D1 environments. This confirms that flat vector representations fail to preserve the relational scheduling information required for robust structural generalization.

In contrast, both GCN and GAT architectures maintain relatively stable performance across increasingly dense scheduling environments and previously unseen workflow structures. Across all evaluated environments, graph-based schedulers consistently remain close to heuristic scheduling performance despite substantial increases in DAG density, dependency connectivity, and action-space complexity.

Second, the scaling experiments demonstrate that graph neural schedulers remain stable as workflow complexity increases from D1 to D3 environments. Although average makespan slightly increases under denser scheduling conditions, the degradation remains limited compared to the dramatic generalization failure observed with MLP-based approaches.

Moreover, the non-deterministic deviations of the GAT policy combined with environment dynamics occasionally result in scheduling trajectories superior to the heuristic baseline on specific unseen workflow configurations.

Although the average GAT performance remains slightly below the heuristic baseline, the number of scenarios outperforming heuristic scheduling suggests that attention-based graph representations may introduce beneficial scheduling variations under certain DAG topologies and dependency structures.

These observations suggest that future work should investigate reinforcement learning approaches capable of optimizing scheduling policies beyond heuristic imitation while preserving graph-aware structural representations.

VI. Discussion

The experimental results demonstrate that graph-based neural scheduling architectures provide significantly stronger structural generalization capabilities than classical dense neural networks for DAG scheduling problems.

The previous MLP-based scheduling approach exhibited severe performance degradation on previously unseen workflow structures, reaching an average makespan of 146.98 on unknown D1 environments. In contrast, the proposed GCN scheduler maintained performance close to the heuristic baseline even when evaluated on entirely unseen DAG topologies.

These observations suggest that preserving workflow topology is essential for robust scheduling generalization. Unlike MLP-based architectures relying on flattened feature representations, Graph Neural Networks (GNNs) explicitly propagate information through dependency relationships during message-passing operations.

As a result, the proposed GCN architecture is capable of learning:

- local task dependencies,
- neighborhood scheduling structures,
- dependency propagation patterns,
- critical task interactions.

This graph-aware inductive bias appears particularly important in DAG scheduling problems where execution order, dependency chains, and workflow topology directly influence scheduling quality.

The experiments additionally indicate that GCN architectures remain relatively stable as workflow density and scheduling complexity increase. Across the D1, D2, and D3 environments, the proposed scheduler consistently maintained performance close to the heuristic scheduling policy despite:

- denser DAG structures,
- larger processor graphs,
- increased dependency connectivity,
- larger combinatorial action spaces.

These observations suggest that graph convolution operations scale effectively on increasingly dense scheduling environments while preserving stable optimization behavior.

The introduction of valid action masking also played a critical role in stabilizing scheduling inference. Without masking, the raw action space contains a very large number of invalid scheduling actions, especially in dense DAG environments where only a small subset of task-CPU assignments satisfies execution constraints.

By restricting inference to executable scheduling actions only, the masking strategy prevents impossible scheduling decisions while preserving the learning objective of the model.

Importantly, the masking mechanism does not provide additional scheduling knowledge to the scheduler itself. The model must still learn:

- which task should be selected,
- which CPU should execute the task,
- under which scheduling context the decision is optimal.

The experiments involving Graph Attention Networks (GATs) provide additional insights regarding topology-aware scheduling mechanisms. Although the average GAT performance remained slightly below the heuristic baseline on large-scale evaluations, the attention-based architecture occasionally produces scheduling trajectories outperforming the heuristic baseline on specific unseen workflow configurations.

In particular, the GAT model outperformed the heuristic scheduler on:

- $27/50$ unseen D3 environments,
- $116/500$ large-scale unseen scenarios.

Unlike GCNs, which aggregate neighborhood information uniformly, GATs dynamically weight neighborhood importance during message propagation. This mechanism may produce slightly different propagation dynamics and embedding distributions across unseen DAG structures.

As a result, generalization noise within the attention mechanisms occasionally leads to scheduling trajectories that outperform the heuristic baseline on specific workflow configurations.

Since the heuristic policy mainly relies on local dependency counts and local CPU congestion estimates, the graph-based schedulers may capture more global structural scheduling patterns through message propagation across the DAG.

However, the experiments also reveal an important limitation of supervised behavior cloning approaches. Since the proposed models are trained directly from heuristic scheduling trajectories, the learned policy remains fundamentally constrained by the quality of the heuristic teacher policy used during dataset generation.

Although graph-based architectures successfully learn meaningful scheduling patterns and generalize effectively across unseen DAG structures, they do not consistently surpass the heuristic baseline in average performance. This limitation is expected in imitation learning settings where the objective remains policy reproduction rather than policy optimization.

VII. Limitations

Despite the promising generalization capabilities demonstrated by graph-based scheduling architectures, several limitations remain in the current work.

First, the proposed models rely entirely on supervised behavior cloning from heuristic scheduling trajectories. As a result, the learned policies remain fundamentally constrained by the quality and biases of the heuristic teacher policy used during dataset generation.

Second, the scheduling environments remain simplified compared to real-world distributed systems. The current simulation does not model:

- communication latency variability,
- memory bottlenecks,
- processor failures,
- dynamic runtime task arrivals,
- heterogeneous GPU resources,
- energy consumption constraints.

Third, the proposed action-space formulation still relies on valid action masking in order to avoid invalid task-CPU assignments during inference. Although effective, this masking strategy highlights the difficulty of learning scheduling policies in extremely sparse combinatorial action spaces.

Although Graph Neural Networks can theoretically process workflow graphs of variable size, the current architecture remains partially constrained by the static linear prediction layer used after graph pooling operations. In particular, the final action layer assumes fixed upper bounds on the number of tasks and processors through predefined dimensions such as `MAX_TASKS` and `MAX_CPUS`.

Consequently, the current implementation cannot directly scale beyond these predefined limits without modifying the architecture itself. Supporting arbitrarily large scheduling environments would require fully dynamic action-generation mechanisms capable of directly evaluating task and processor embeddings during inference rather than relying on fixed-size action vectors.

Finally, although the D3 environments significantly increase DAG density and scheduling complexity, the evaluated environments remain substantially smaller than real-world cloud or HPC scheduling infrastructures.

VIII. Future Work

Future work should investigate reinforcement learning approaches capable of optimizing scheduling policies beyond heuristic imitation.

In particular, combining Graph Neural Networks with policy optimization methods such as Proximal Policy Optimization (PPO) could allow schedulers to directly learn adaptive scheduling strategies through environment interaction rather than heuristic reproduction alone.

An important limitation of the current architecture lies in the use of a global pooling operation followed by a static linear prediction layer over the complete action space. Although effective for behavior cloning, this design fixes the action-space dimensionality and does not directly evaluate task embeddings individually during scheduling inference.

Future scheduling architectures should therefore investigate node-level scheduling mechanisms capable of directly scoring task embeddings without compressing the entire DAG into a single global representation. Potential approaches include:

- node-level task scoring,
- pointer-network scheduling mechanisms,
- graph transformers for scheduling,
- heterogeneous CPU-GPU scheduling,
- distributed cloud scheduling,
- online adaptive scheduling,

- energy-aware scheduling optimization.

Such architectures may improve scalability, structural generalization, and dynamic scheduling adaptability on large-scale heterogeneous DAG environments.

IX. Conclusion

This work investigated graph-based neural architectures for DAG scheduling problems in heterogeneous multiprocessor environments.

The experiments demonstrate that Graph Neural Networks provide substantially stronger structural generalization capabilities than classical dense MLP architectures on unseen workflow topologies. In particular, the proposed GCN and GAT schedulers successfully preserve scheduling performance across increasingly dense DAG environments while maintaining stable optimization behavior.

The results suggest that graph-aware message propagation constitutes a substantially more suitable representation learning framework for DAG scheduling problems than flattened vector representations.

However, the experiments also highlight an important limitation of supervised behavior cloning approaches. Since the proposed models directly imitate heuristic scheduling trajectories, their performance remains fundamentally constrained by the heuristic teacher policy used during dataset generation.

Future work should therefore investigate reinforcement learning approaches capable of optimizing scheduling decisions beyond heuristic performance ceilings while preserving graph-aware structural representations.

References

[1] L. Lejaille, "Task scheduling on multi-cpu graph architectures using supervised policy learning," 2026. [Online]. Available: https://github.com/LLEJAILLE/Task-Scheduling-M-CPU-MLP

[2] M. Hussain, "A heuristic approach to spark workflow task scheduling on heterogeneous nodes," *Futur generation computer systems*, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X25006296?ref=pdf_download&fr=RR-2&rr=a0178f9a9e0c5ce6

[3] X. Zhang, "An enhanced priority-based scheduling heuristic for dag applications with temporal unpredictability in task execution and data transmission," *Futur generation computer systems*, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X18311919

[4] S. Rahim, "A survey of machine learning-driven task scheduling approaches for multiprocessor systems," *Journal of Systems Architecture*, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762125003005?ref=pdf_download&fr=RR-2&rr=a0179f17b8ae5ce6

[5] W. Gao, "Gnn-based deep reinforcement learning for computation task scheduling in autonomous multi-robot systems," *Journal of Systems Architecture*, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762125002061?ref=pdf_download&fr=RR-2&rr=a0179f6e7eb65ce6

[6] A. HATTAY, "On the role of dag topology in energy-aware cloud scheduling : A gnn-based deep reinforcement learning approach," *arxiv*, 2026. [Online]. Available: https://arxiv.org/pdf/2604.09202

[7] B. Deng, "Mutual gnn-mlp distillation for robust graph adversarial defense," *Neural Networks*, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608025003922?ref=pdf_download&fr=RR-2&rr=a017a6303ad75ce6